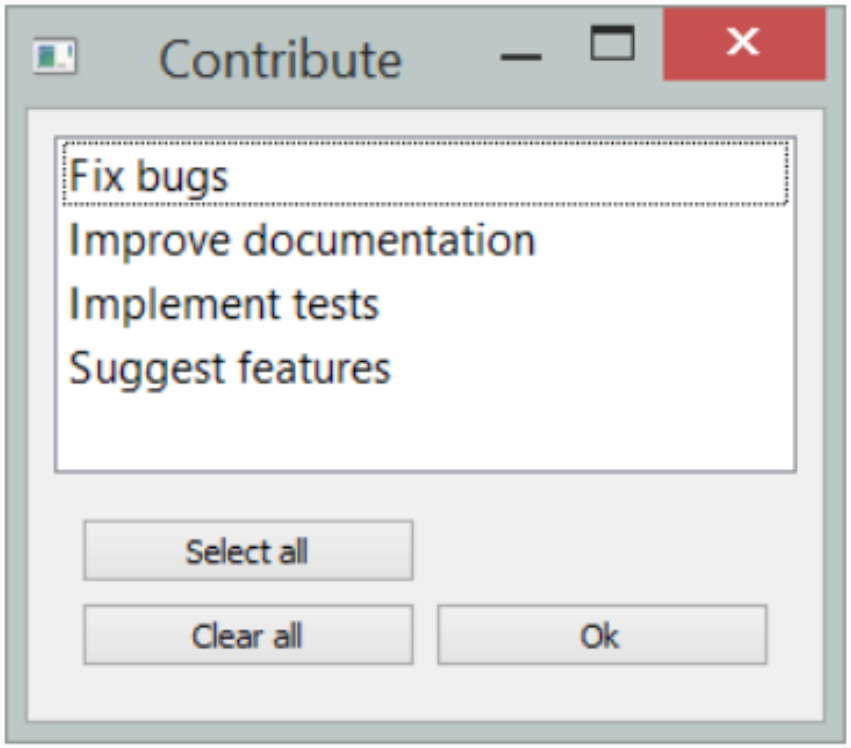
EasyGUI_Qt

Inspired by EasyGUI, designed for PyQt



Interfícies gràfiques d'usuari amb pop-ups

https://easygui-qt.readthedocs.io/en/latest/index.html

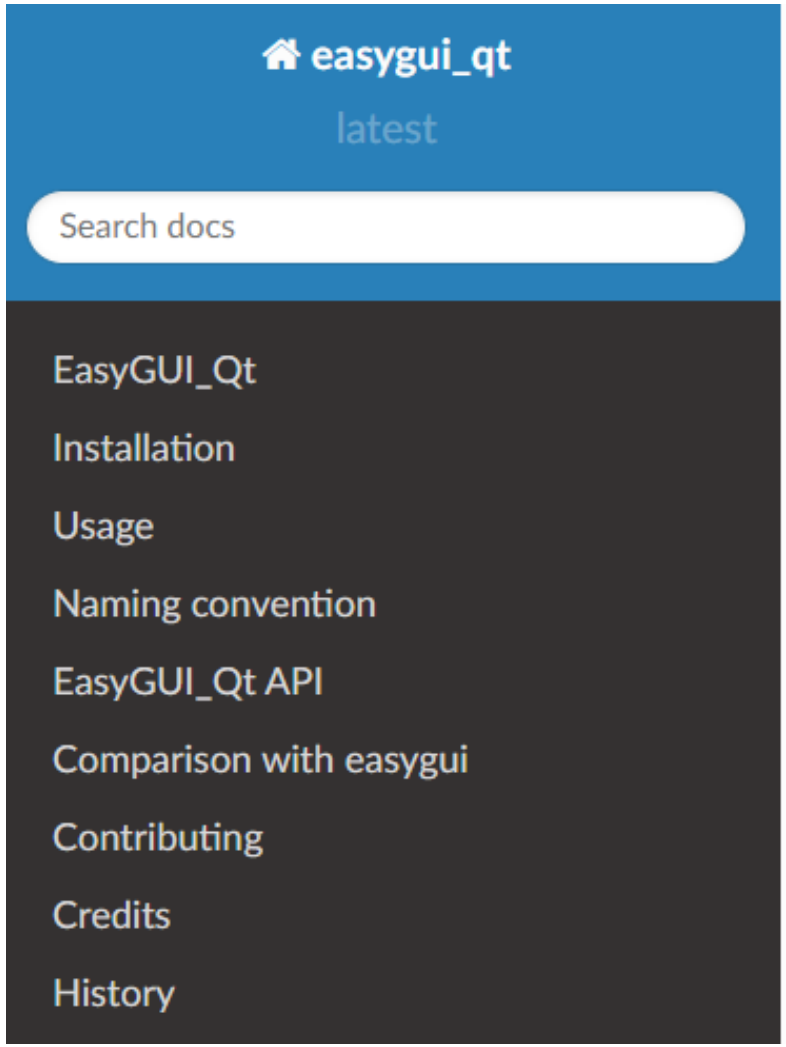
Unió Europea
Fons social europeu

Característiques principals

1. Permeten lògica seqüencial (no distribuïda)
2. Faciliten l'aprenentatge
    - poques funcions
    - molta operativa per defecte: aspecte i situació dels elements
3. Grans limitacions (només emergents, no apte per aplicacions de gestió)

Ho estudiarem per veure aplicacions pràctiques de cadenes, llistes i iniciar-nos en les funcions.

https://easygui-qt.readthedocs.io/en/latest/index.html



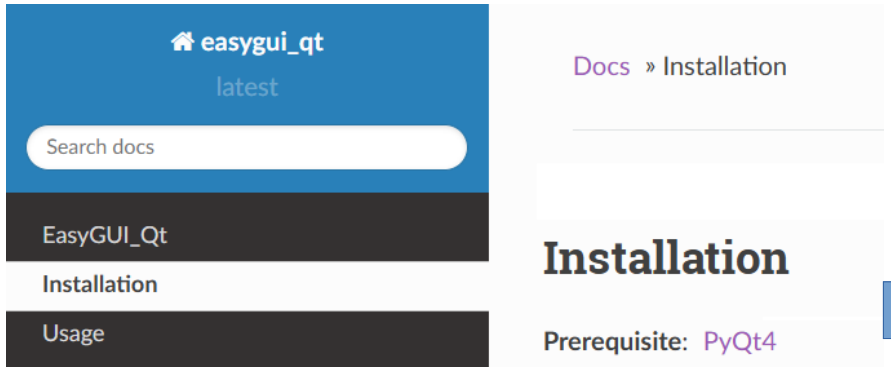
Welcome to easygui_qt's documentation!

Contents:

- EasyGUI_Qt
  - Python version
  - Design philosophy
  - Roadmap
  - Similar projects
- Installation
- Usage
  - Demos
- Naming convention
  - Specifying arguments

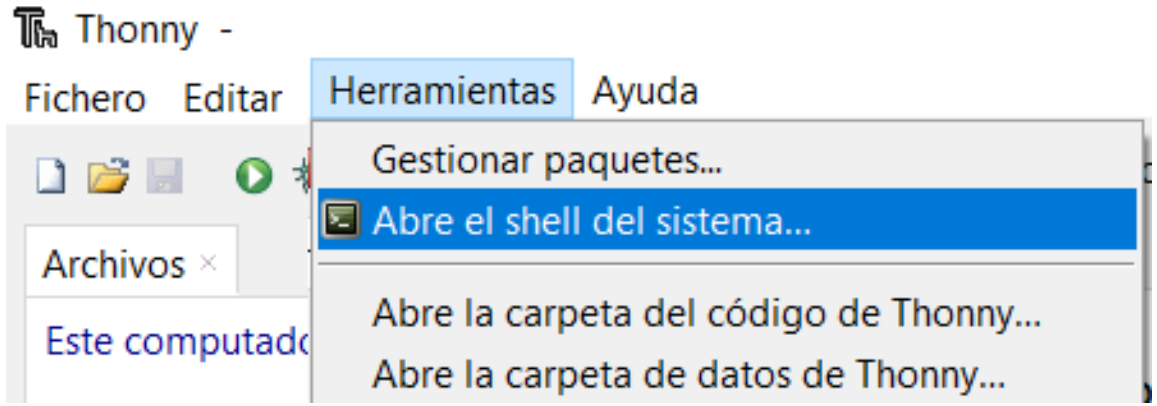
Aquí trobem la documentació oficial

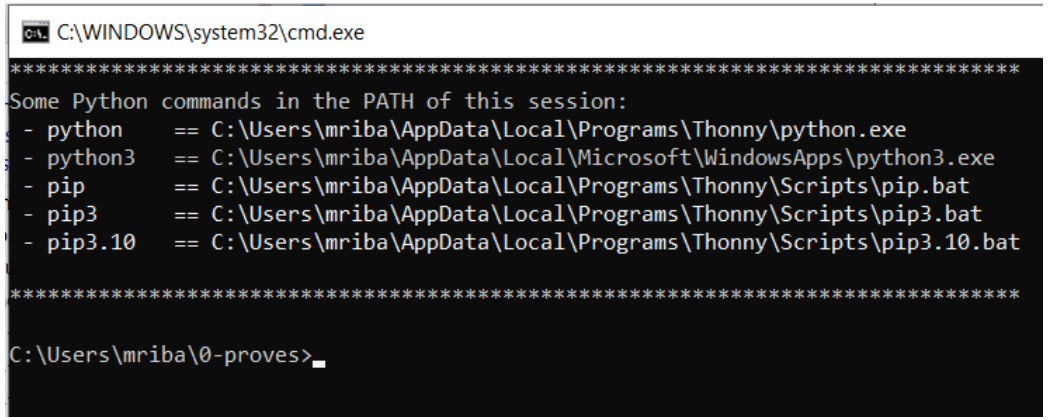
https://easygui-qt.readthedocs.io/en/latest/installation.html



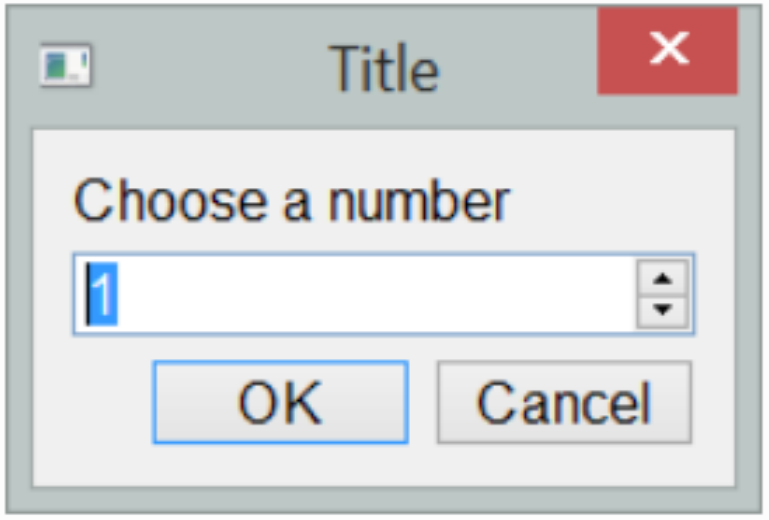
Així instal·lem les llibreries al nostre entorn local.

pip install PyQt5
pip install easygui_qt



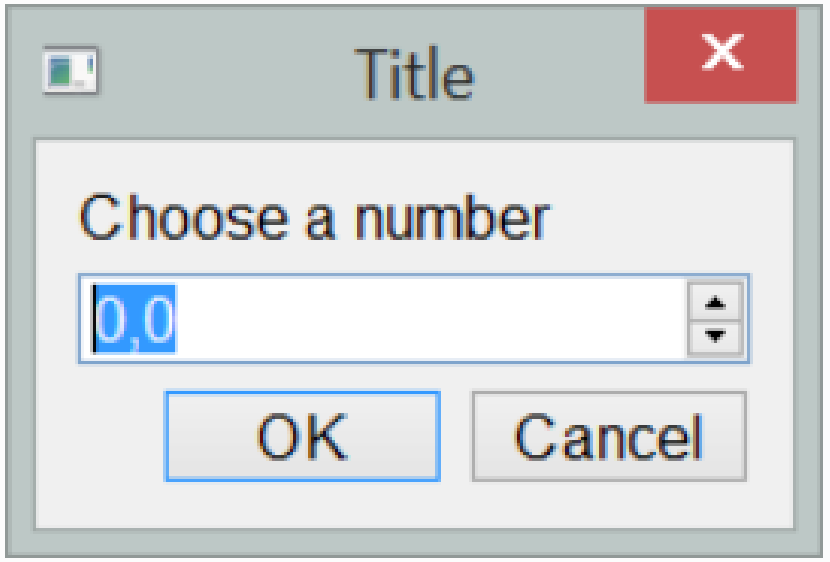


```
>>> import easygui_qt as easy
>>> number = easy.get_int()
```



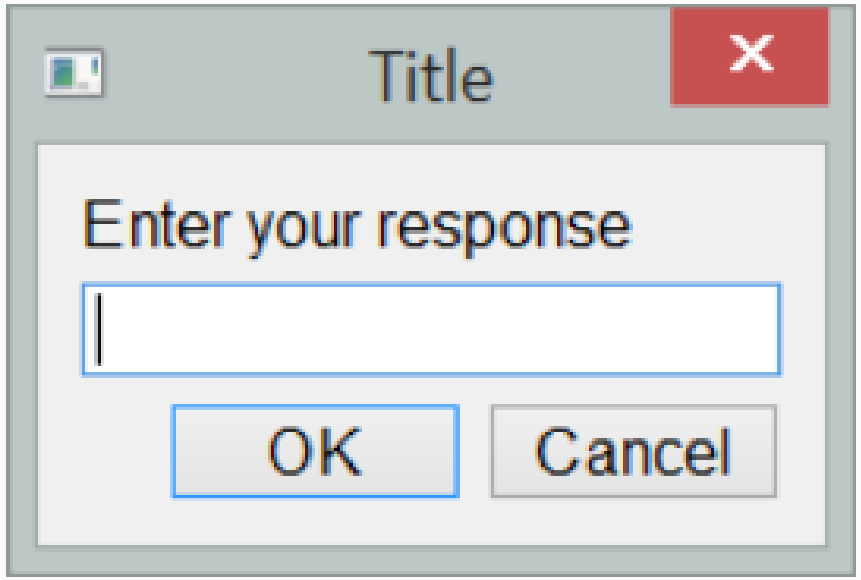
get_int (*message='Choose a number', title='Title', default_value=1, min_=0, max_=100, step=1*)

```
>>> import easygui_qt as easy
>>> number = easy.get_float()
```

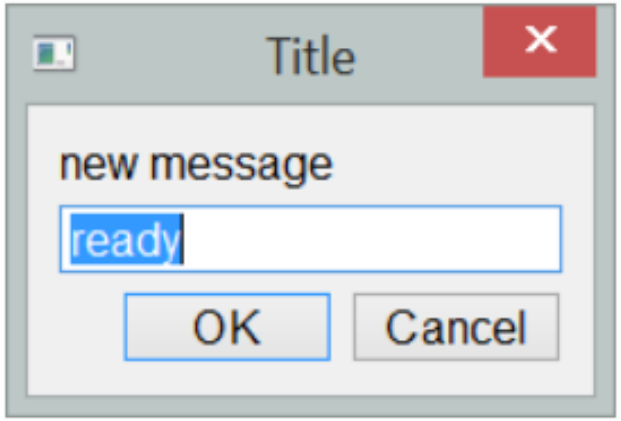


get_float (*message='Choose a number', title='Title', default_value=0.0, min_=-10000, max_=10000, decimals=3*)

```
>>> import easygui_qt as easy
>>> reply = easy.get_string()
```

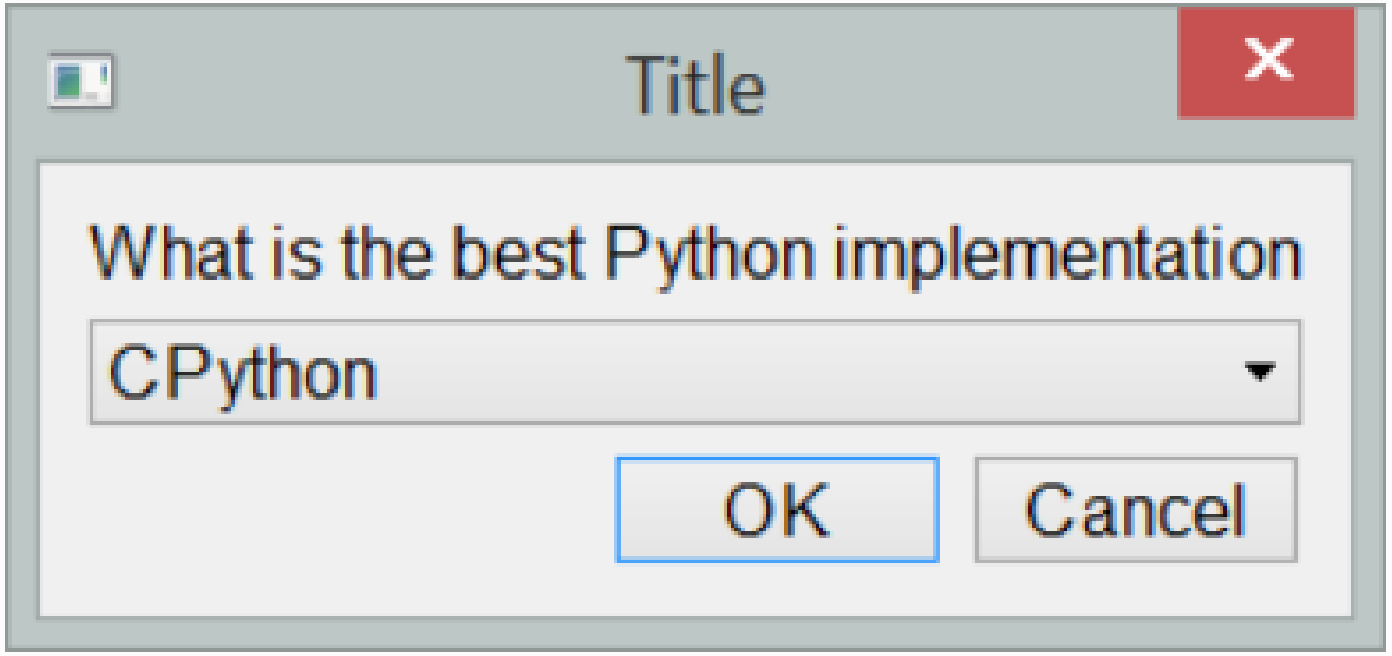



```
>>> reply = easy.get_string("new message", default_response="ready")
```



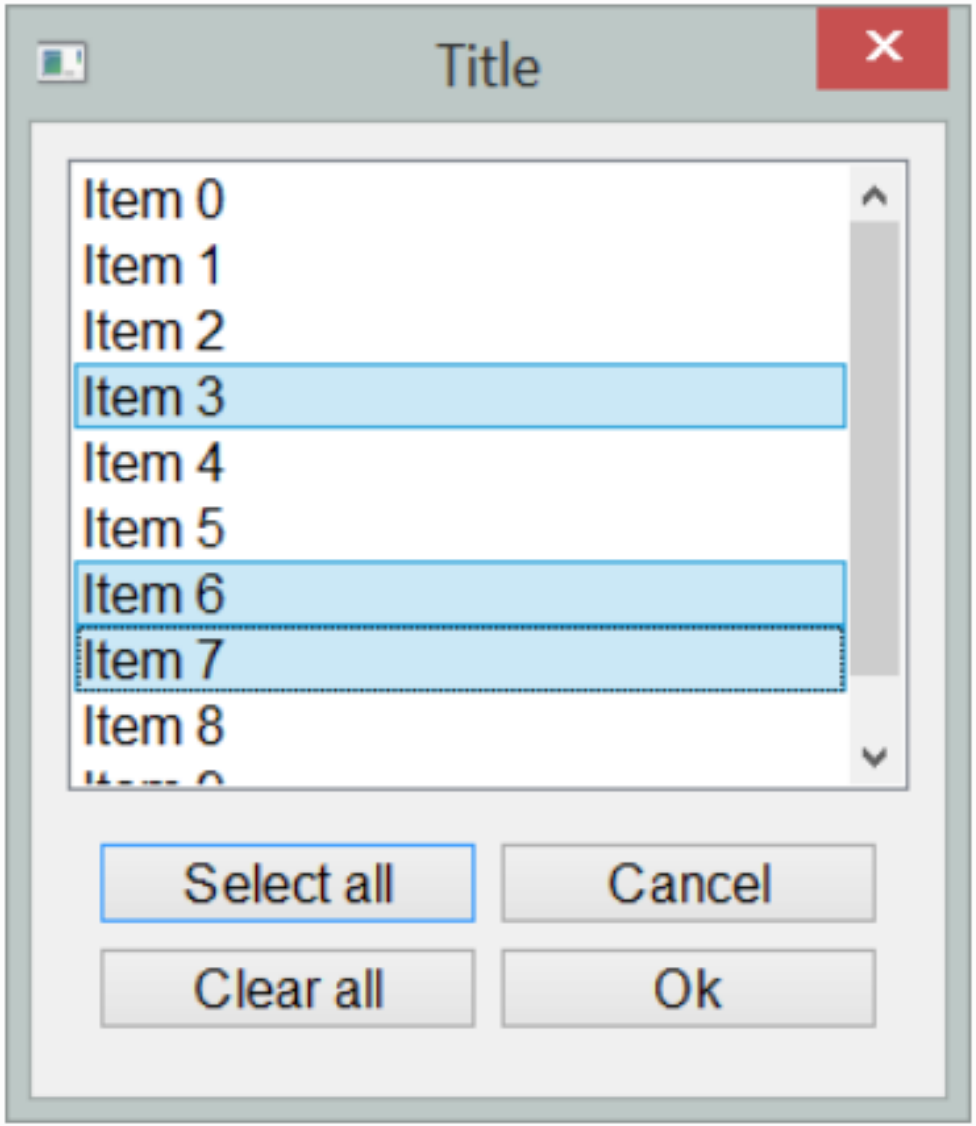
get_string (*message='Enter your response', title='Title', default_response=''*)

```
>>> import easygui_qt as easy
>>> choices = ["CPython", "Pypy", "Jython", "IronPython"]
>>> reply = easy.get_choice("What is the best Python implementation",
...                         choices=choices)
```



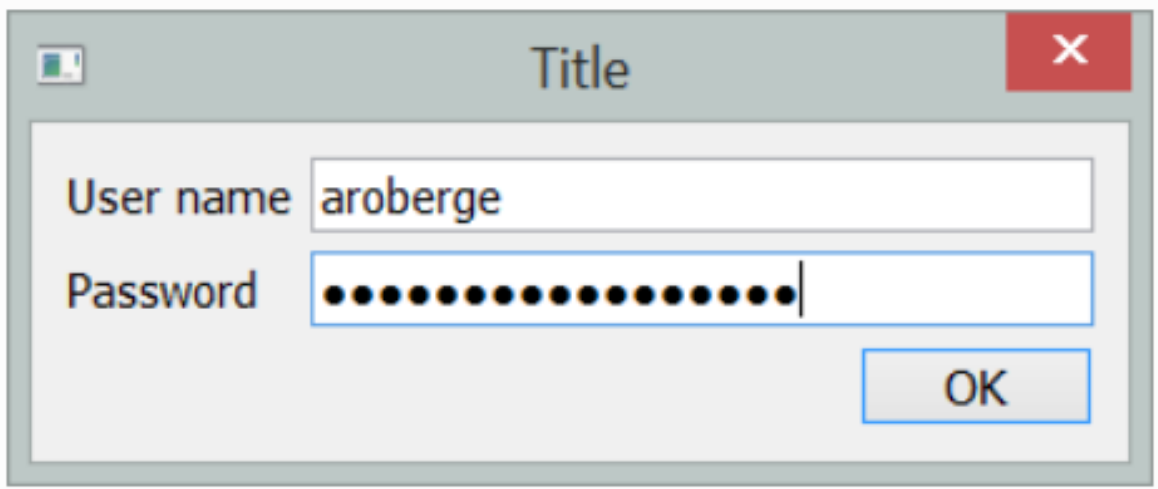
On està la llista ? La cadena ? La funció ? Els paràmetres ?

```
>>> import easygui_qt as easy
>>> choices = easy.get_list_of_choices()
```



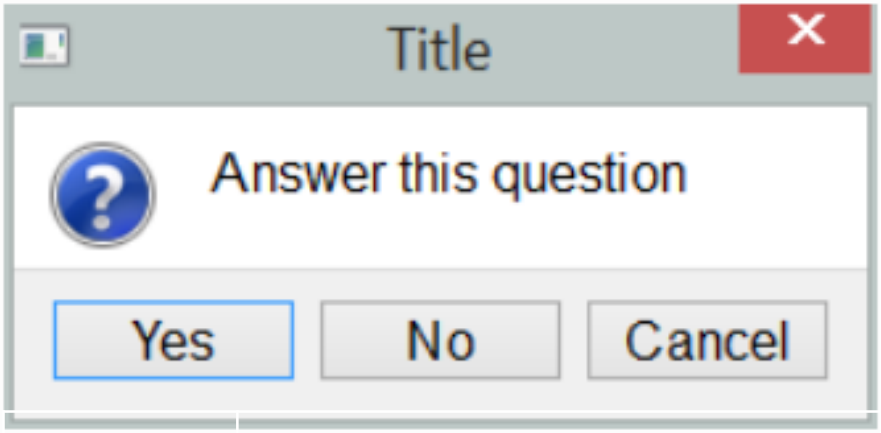
get_many_strings(*title='Title', labels=None, masks=None*)

```
>>> import easygui_qt as easy
>>> labels = ["User name", 'Password']
>>> masks = [False, True]
>>> reply = easy.get_many_strings(labels=labels, masks=masks)
>>> reply
OrderedDict([('User name', 'aroberge'), ('Password', 'not a good password')])
```



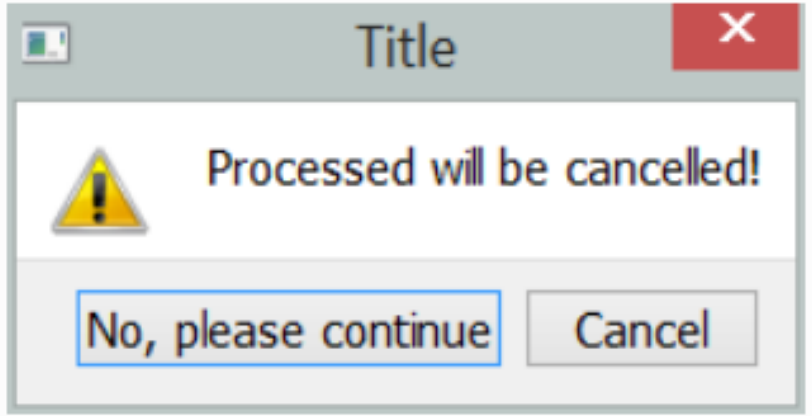
get_yes_or_no (*message='Answer this question', title='Title'*)

```
>>> import easygui_qt as easy
>>> choice = easy.get_yes_or_no()
```



Returns: True for “Yes”, False for “No”, and None for “Cancel”.

```
>>> import easygui_qt as easy
>>> choice = easy.get_continue_or_cancel()
```



True for “Continue”, False for “Cancel”

get_continue_or_cancel (*message='Processed will be cancelled!', title='Title', continue_button_text='Continue', cancel_button_text='Cancel'*)

Pràctica 06.P00:

1. Fes un programa que demani un número enter i mostri un missatge indicant si es parell/par o senar/impar

2. Fes un programa que permeti escollir un arxiu python, navegant amb l'explorador i després el mostri com a codi python.

3. Fes un programa que demani un color amb un selector de color i mostri quant de red, green, blue té.

4. Fes un programa que demani una data usant un calendari, i després mostri en una finestra la data en format dd-mm-aaaa

Pràctica 06.P01: Millorar exercicis antics

Enunciat

Escull un exercici fets el dia abans quan treballàvem els bucles while. Modifica'l per tal que les entrades de dades i les visualitzacions estiguin fetes amb easy_GUI_qt.

Per exemple : de la sessió 06, la pràctica P05.4

```
cars =[]

print ("MENU PARKING COCHES 1.Entra, 2.Sale, 3.Lista, 0.Salir")
```

```
opcion = ""
while opcion != 0 :
    opcion = int(input("Introduce opcion:"))
    if opcion == 0 :
        continue
    elif opcion == 3 :
        print (cars)
    else :
        matricula = input ("Introduzca la matrícula: ")
        if opcion == 1:
            cars.append(matricula)
        else :
            cars.remove(matricula)
```

Pràctica 06.P02 : LogAccessos.py

Manel, 260898
Neus, carvana89

Enunciat

Obre un arxiu d'usuaris que conté el nom d'usuari i la contrasenya "usuaris.dat".

Demana usuari i contrasenya, i determina si està o no a l'arxiu d'usuaris :

- si està, i la contrasenya és vàlida, informa: «Accés autoritzat», mostra una imatge d'un escriptori windows i acaba el programa.
- si la contrasenya és incorrecta, informa : «Contrasenya incorrecte», i torna a demanar.
- si l'usuari és invàlid informa : «Usuari invàlid», i torna a demanar.
- permet un màxim de 3 intents infructuosos i si no s'han pogut identificar correctament, acaba el programa.

Manel, 260898
Neus, carvana89

Pseudocodi

- obre arxiu “usuaris.dat”, llegeix el contingut i tanca l’arxiu.
- fes 3 cops:
  - demana usuari / contrasenya
  - si usuari està a la llista d’usuaris ...
    - si la contrasenya és correcta ...
      - mostra missatge «Accés autoritzat»
      - mostra imatge «escriptori.jpg»
      - surt del bucle
    - sinó (la contrasenya és incorrecte)
      - mostra missatge «Contrasenya incorrecte»
  - sinó (l’usuari no està a la llista)
    - mostra missatge «Usuari invàlid»

Pràctica 06.P03 : Comandes

Enunciat

Volem fer un petit aplicatiu per tal que varis operadors puguin crear comandes de treballs de pintura. Primer cal identificar l'operador amb usuari i contrasenya i si l'usuari és correcte pot continuar creant comandes.

Per cada comanda es demana :

- nom, telèfon, correu, tipus de treball, color, data de servei
- Un cop acceptades les dades es gravarà un registre a l'arxiu comandes.csv

Pseudocodi

demanar dades

- valida_usuari (veure funcions a l'apartat d'ajuda)
- Si usuari no es correcte surt del programa

Cicle de comandes

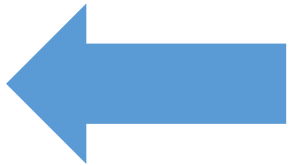
- mentre no vulguin sortir :
    - demana nom, telèfon, correu
    - demana tipus de treball
    - demana color (amb selector hexadecimal)
    - Demana la data de servei
    - Grava les dades en un arxiu csv
- Tanca l'arxiu

Ajudes

```
#---------------------- Inici del programa

if valida_usuari():
    comanda = demana_comanda()
    grava_comanda(comanda)


#---------------------- Fi del programa
```



En la primera versió fes que només demani una comanda.
Quan et funcioni tot, ja afegiràs el bucle while.

Ajudes

```
def valida_usuari():

    f = open("./dat/usuaris.csv", "r", encoding="UTF-8")
    linees = f.read().split("\n")
    f.close()

    usuari, pwd = demana_usuari()
    for linea in linees :
        if linea.find(usuari) > -1 :
            if pwd == linea.split(";")[1]:
                return (True)
            else :
                print("Error contrasenya")
                return(False)
    print("Error usuari")
    return False
```

Ajudes

```
def demana_usuari():

    labels = ["Usuari", "Contrasenya"]
    masks = [False, True]
    dades = easy.get_many_strings(labels=labels, masks=masks)

    return dades["Usuari"], dades["Contrasenya"]
```

Ajudes

```
def demana_comanda():
    labels = ["Nom", "Telèfon", "Correu"]
    dades = easy.get_many_strings(labels=labels)
    comanda = [dades['Nom'], dades['Telèfon'], dades['Correu']]

    productes = ['pintura exterior', 'pintura interior']
    producte = easy.easygui_qt.get_choice(message='Escull el producte', choices=productes)
    comanda.append(producte)

    m2 = easy.get_int(message="quants m2")
    comanda.append(str(m2))

    color = easy.get_color_rgb()
    comanda.append(str(color))

    easy.set_language('es')
    date = easy.get_date()
    comanda.append(str(date))

    return(comanda)
```

Ajudes

```
def grava_comanda(comanda):
    #"nom, telèfon, correu, producte, m2, color, data"
    print(",".join(comanda))
```

Pràctica 06.P04 : Pulsacions.py

Enunciat

Feu un programa que generi aleatòriament* les pulsacions d'una persona durant un dia escollit i les desi en un lloc seleccionat per l'usuari.

Cal demanar el nom i el també el perfil de la persona (estudiant, obrer, DJ) i per generar patrons aleatoris coherents amb les activitats típiques d'aquests perfils.

Els registres generats inclouran el nom, el perfil, l' hora (des de les 00 a les 23) i les pulsacions mitjanes cada mig hora des de les 7:00 a les 23:00.

A l'inici, el programa demanarà en quina carpeta es vol gravar l'arxiu i el nom de l'arxiu es calcularà amb el nom per la persona (sense espais i camelCase) junt amb la data en format any mes dia : rosaLopez-2025-07-01.dat

Pseudocodi

demanar dades
- demana nom
- demana perfil (estudiant, obrer, DJ)
- demana data
- demana ubicació de l'arxiu

calcular dades
- recalcula el nom passant a CamelCase i treient els espais
- posa la data en format aaaa-mm-dd
- obre l'arxiu (usant el nom-data.dat) en la ubicació proporcionada
- per cada hora des de les 00 a les 23
  - fes dos cops ...
    - calcula pulsacions segons el patró que has decidit pel perfil
    - grava registre usant : perfil ; hora; pulsacions

Segueix el pseudocodi

desar dades
- tanca l'arxiu
- mostra el missatge «procés finalitat, arxiu generat *ubicació+nom arxiu*»

Possible exemple de patrons :

```
DJ  = {0:120, 1:100, 2:100, 3:100, 4:100, 5:60, 7:60, 8:60, 9:60, 10:60, 11:60,
12:60, 13:60, 14:80, 15:80, 16:90, 17:120, 18:120, 19:120, 20:100, 21:80, 22:80,
23:100}
```

Solució Pràctica 06.P01

```
import easygui_qt as easy

cars =[]

opciones = "1.Entra,2.Sale,3.Lista,0.Salir".split(",")

opcion = ""
while opcion != 0 :
    opcion = easy.get_choice(message='opción', title='Menú Parking', choices=opciones)
    opcion = int(opcion[0])
    if opcion == 0 :
        continue
    elif opcion == 3 :

        easy.show_text("Coches en el parking", "\n".join(cars))
        print (cars)
    else :
        matricula = easy.get_string(message="matrícula")
        if opcion == 1:
            cars.append(matricula)
        else :
            cars.remove(matricula)
print("Fin del programa")
```

Solució Pràctica 06.P03

```
def valida_usuari():

    f = open("./dat/usuaris.csv", "r", encoding="UTF-8")
    linees = f.read().split("\n")
    f.close()

    usuari, pwd = demana_usuari()
    for linea in linees :
        if linea.find(usuari) > -1 :
            if pwd == linea.split(";")[1]:
                return (True)
            else :
                print("Error contrasenya")
                return(False)
    print("Error usuari")
    return False

def demana_usuari():

    labels = ["Usuari", "Contrasenya"]
    masks = [False, True]
    dades = easy.get_many_strings(labels=labels, masks=masks)

    return dades["Usuari"], dades["Contrasenya"]
```

```
def demana_comanda():
    labels = ["Nom", "Telèfon", "Correu"]
    dades = easy.get_many_strings(labels=labels)
    comanda = [dades['Nom'], dades['Telèfon'], dades['Correu']]

    productes = ['pintura exterior', 'pintura interior']
    producte = easy.easygui_qt.get_choice(message='Escull el producte', choices=productes)
    comanda.append(producte)

    m2 = easy.get_int(message="quants m2")
    comanda.append(str(m2))

    color = easy.get_color_rgb()
    comanda.append(str(color))

    easy.set_language('es')
    date = easy.get_date()
    comanda.append(str(date))

    return(comanda)
```

```
def grava_comanda(comanda):
    #"nom, telèfon, correu, producte, m2, color, data"
    print(",".join(comanda))
#---------------------- Inici del programa

if valida_usuari():
    comanda = demana_comanda()
    grava_comanda(comanda)
```